LAPORAN PRAKTIKUM
Implementasi Function dan Trigger

Tugas Terstruktur
Rekaya Arsitektur dan Basis Data

Dosen Pengampu
Cecep Kurnia Sastradipraja, S.Kom., M.Kom.

Disusun Oleh:
Nurul Karimah 2507065

Sistem Informasi B

INSTITUT TEKNOLOGI GARUT
PROGRAM STUDI SISTEM INFROMASI
2026

1. Tujuan

Praktikum ini bertujuan menguji stored function dan trigger yang telah dibuat pada Praktikum 1 sampai 3, yaitu function hitung_total dan hitung_diskon, serta trigger validasi_penjualan dan update_stok_penjualan, pada studi kasus sistem penjualan toko elektronik.

2. Deskripsi Kasus dan Rancangan Solusi

Sebuah toko elektronik ingin mengotomatisasi proses penjualan dan pengendalian persediaan. Setiap produk memiliki identitas, nama, harga, dan jumlah stok. Ketika transaksi dilakukan, sistem harus menghitung total harga, memvalidasi stok, memperbarui persediaan, dan menyimpan catatan perubahan stok. Basis data yang digunakan bernama praktikum_function_trigger dengan tiga tabel: produk, penjualan, dan log_stok.

Rancangan solusi yang diuji pada tugas ini ditunjukkan pada tabel berikut.

Objek	Jenis	Fungsi
hitung_total	Function	Menghitung total transaksi (harga satuan x jumlah).
hitung_diskon	Function	Memberikan diskon 10% untuk total transaksi minimal Rp1.000.000, selain itu diskon 0.
validasi_penjualan	Trigger BEFORE INSERT	Memvalidasi jumlah, harga, keberadaan produk, dan ketersediaan stok sebelum penjualan disimpan, serta mengisi kolom total.
update_stok_penjualan	Trigger AFTER INSERT	Mengurangi stok produk dan mencatat stok awal dan stok akhir ke tabel log_stok.

Lingkungan praktikum: layanan MySQL dijalankan melalui DBngin (MySQL 9.7.1, port 33011) dan perintah SQL dijalankan melalui DBeaver.



Gambar 1. DBngin: instance MySQL aktif (port 33011)

3. Kode SQL Lengkap

Kode SQL berikut dijalankan melalui DBeaver. Bagian A membuat function dan trigger, sedangkan bagian B berisi pengujian.

```
-- ===== BAGIAN A: PEMBUATAN FUNCTION DAN TRIGGER =====
USE praktikum_function_trigger;

-- PRAKTIKUM 2 - STORED FUNCTION

DELIMITER //

-- Function menghitung total = harga satuan x jumlah
CREATE FUNCTION hitung_total(
    p_harga DECIMAL(12,2),
```

```
    p_jumlah INT
)
RETURNS DECIMAL(14,2)
DETERMINISTIC
NO SQL
BEGIN
    RETURN p_harga * p_jumlah;
END //

-- Function diskon 10% untuk transaksi minimal Rp1.000.000
CREATE FUNCTION hitung_diskon(
    p_total DECIMAL(14,2)
)
RETURNS DECIMAL(14,2)
DETERMINISTIC
NO SQL
BEGIN
    IF p_total >= 1000000 THEN
        RETURN p_total * 0.10;
    ELSE
        RETURN 0;
    END IF;
END //


-- PRAKTIKUM 3 - TRIGGER

-- Trigger validasi sebelum data penjualan disimpan
CREATE TRIGGER validasi_penjualan
BEFORE INSERT ON penjualan
FOR EACH ROW
BEGIN
    DECLARE v_stok INT DEFAULT NULL;
    DECLARE v_ada INT DEFAULT 0;

    IF NEW.jumlah <= 0 THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT =
            'Jumlah pembelian harus lebih dari nol';
    END IF;

    IF NEW.harga_satuan <= 0 THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT =
            'Harga satuan harus lebih dari nol';
    END IF;

    SELECT COUNT(*)
    INTO v_ada
    FROM produk
    WHERE id_produk = NEW.id_produk;

    IF v_ada = 0 THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Produk tidak ditemukan';
    END IF;

    SELECT stok
```

```
    INTO v_stok
    FROM produk
    WHERE id_produk = NEW.id_produk
    FOR UPDATE;

    IF NEW.jumlah > v_stok THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT =
            'Stok produk tidak mencukupi';
    END IF;

    SET NEW.total =
        hitung_total(NEW.harga_satuan, NEW.jumlah);
END //

# Trigger memperbarui stok dan mencatat log setelah penjualan
CREATE TRIGGER update_stok_penjualan
AFTER INSERT ON penjualan
FOR EACH ROW
BEGIN
    DECLARE v_stok_awal INT;
    DECLARE v_stok_akhir INT;

    SELECT stok
    INTO v_stok_awal
    FROM produk
    WHERE id_produk = NEW.id_produk;

    SET v_stok_akhir = v_stok_awal - NEW.jumlah;

    UPDATE produk
    SET stok = v_stok_akhir
    WHERE id_produk = NEW.id_produk;

    INSERT INTO log_stok (
        id_produk,
        stok_awal,
        stok_akhir,
        keterangan
    )
    VALUES (
        NEW.id_produk,
        v_stok_awal,
        v_stok_akhir,
        'Pengurangan stok karena penjualan'
    );
END //

DELIMITER ;

-- PEMERIKSAAN: pastikan 2 function dan 2 trigger sudah terbuat
SHOW FUNCTION STATUS
WHERE Db = 'praktikum_function_trigger';

SHOW TRIGGERS
FROM praktikum_function_trigger;

-- ===== BAGIAN B: PENGUJIAN =====
```

```
-- ===== TUGAS 1 =====
SELECT hitung_total(250000, 4) AS total_transaksi;

-- ===== TUGAS 2 =====
-- Pengujian A
SELECT 800000 AS total_transaksi,
       hitung_diskon(800000) AS diskon,
       800000 - hitung_diskon(800000) AS total_bayar;

-- Pengujian B
SELECT 1500000 AS total_transaksi,
       hitung_diskon(1500000) AS diskon,
       1500000 - hitung_diskon(1500000) AS total_bayar;

-- ===== TUGAS 3 =====
-- Skenario A (diterima)
INSERT INTO penjualan (id_produk, jumlah, harga_satuan, total)
VALUES (2, 1, 150000, 0);

-- Skenario B (ditolak)
INSERT INTO penjualan (id_produk, jumlah, harga_satuan, total)
VALUES (2, 100, 150000, 0);

-- Skenario C (ditolak)
INSERT INTO penjualan (id_produk, jumlah, harga_satuan, total)
VALUES (2, 0, 150000, 0);

-- Bukti transaksi yang tersimpan
SELECT * FROM penjualan WHERE id_produk = 2;

-- ===== TUGAS 4 =====
-- Stok awal Keyboard
SELECT id_produk, nama_produk, stok FROM produk WHERE id_produk = 3;

INSERT INTO penjualan (id_produk, jumlah, harga_satuan, total)
VALUES (3, 3, 350000, 0);

-- Stok akhir Keyboard
SELECT id_produk, nama_produk, stok FROM produk WHERE id_produk = 3;

-- Log stok Keyboard
SELECT * FROM log_stok WHERE id_produk = 3;

-- ===== TUGAS 5 =====
-- Stok awal Laptop
SELECT id_produk, nama_produk, stok AS stok_sebelum FROM produk WHERE
id_produk = 1;

INSERT INTO penjualan (id_produk, jumlah, harga_satuan, total)
VALUES (1, 2, 7500000, 0);

-- Total, diskon, dan total bayar
SELECT id_penjualan, total AS total_transaksi,
       hitung_diskon(total) AS diskon,
       total - hitung_diskon(total) AS total_bayar
FROM penjualan
WHERE id_produk = 1
```

```
ORDER BY id_penjualan DESC
LIMIT 1;

-- Stok akhir Laptop
SELECT id_produk, nama_produk, stok AS stok_sesudah FROM produk WHERE
id_produk = 1;

-- Log stok Laptop
SELECT * FROM log_stok WHERE id_produk = 1;
```

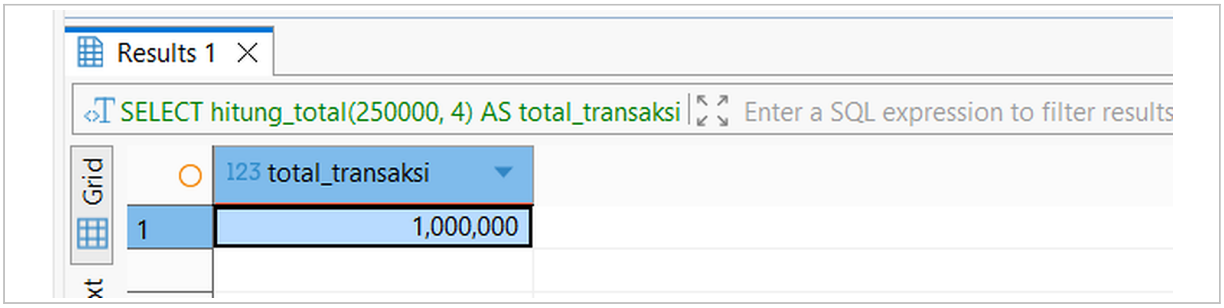
4. Hasil Pengujian

4.1 Tugas 1. Function Perhitungan Total

Function hitung_total diuji dengan harga satuan Rp250.000 dan jumlah 4 unit.

```
SELECT hitung_total(250000, 4) AS total_transaksi;
```

Hasil: total transaksi Rp1.000.000, sesuai dengan target.



Gambar 2. Hasil pengujian function hitung_total(250000, 4)

4.2 Tugas 2. Function Diskon

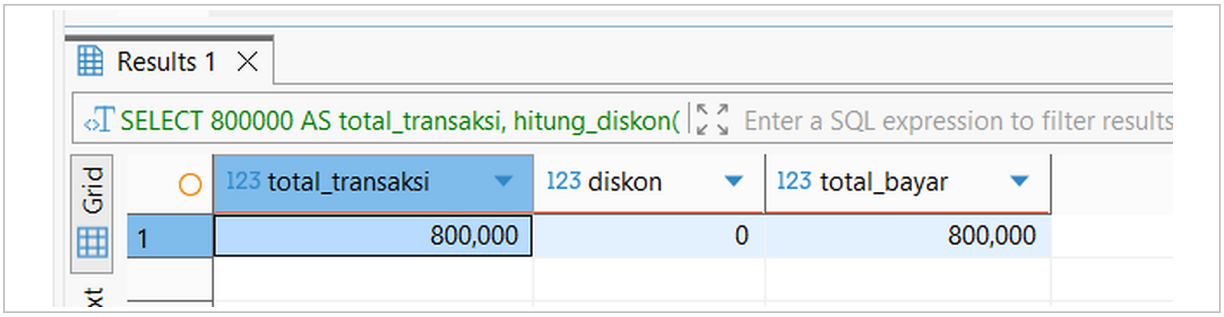
Function hitung_diskon diuji pada dua transaksi, yaitu Rp800.000 (A) dan Rp1.500.000 (B).

```
SELECT 800000 AS total_transaksi,
    hitung_diskon(800000) AS diskon,
    800000 - hitung_diskon(800000) AS total_bayar;

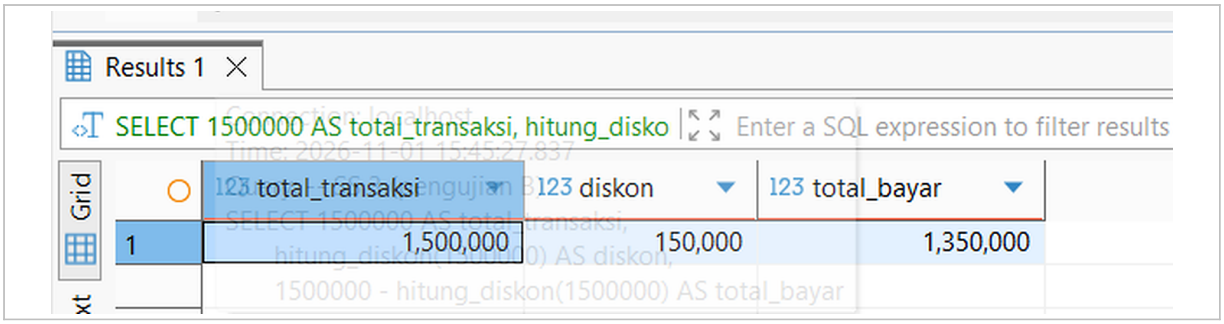
SELECT 1500000 AS total_transaksi,
    hitung_diskon(1500000) AS diskon,
    1500000 - hitung_diskon(1500000) AS total_bayar;
```

Pengujian	Total transaksi	Diskon	Total bayar
A	Rp800.000	Rp0	Rp800.000
B	Rp1.500.000	Rp150.000	Rp1.350.000

Transaksi A tidak memperoleh diskon karena kurang dari Rp1.000.000, sedangkan transaksi B memperoleh diskon 10%.



Gambar 3. Hasil pengujian diskon A: total transaksi Rp800.000



Gambar 4. Hasil pengujian diskon B: total transaksi Rp1.500.000

4.3 Tugas 3. Validasi Transaksi Penjualan

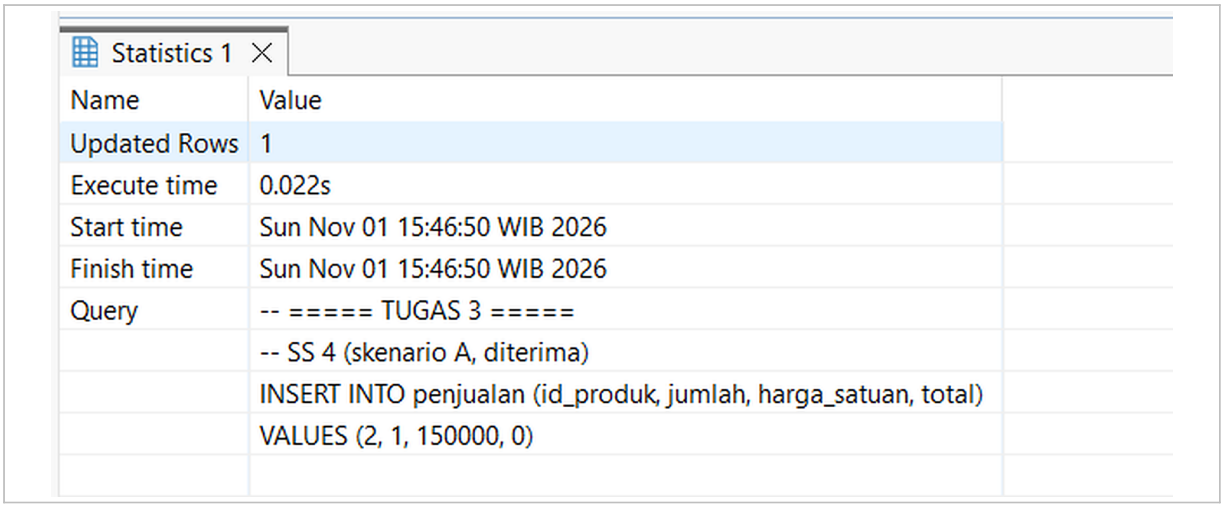
Trigger validasi_penjualan diuji dengan tiga skenario pembelian Mouse. Setiap skenario dijalankan secara terpisah.

```
-- Skenario A (diterima)
INSERT INTO penjualan (id_produk, jumlah, harga_satuan, total)
VALUES (2, 1, 150000, 0);

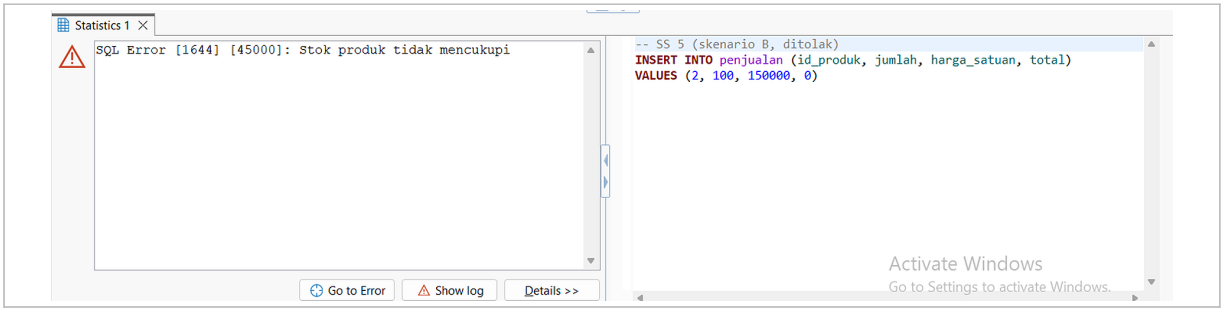
-- Skenario B (ditolak)
INSERT INTO penjualan (id_produk, jumlah, harga_satuan, total)
VALUES (2, 100, 150000, 0);

-- Skenario C (ditolak)
INSERT INTO penjualan (id_produk, jumlah, harga_satuan, total)
VALUES (2, 0, 150000, 0);
```

Skenario	Produk	Jumlah	Hasil
A	Mouse	1	Diterima, data tersimpan
B	Mouse	100	Ditolak: "Stok produk tidak mencukupi"
C	Mouse	0	Ditolak: "Jumlah pembelian harus lebih dari nol"



Gambar 5. Skenario A: INSERT 1 unit Mouse berhasil (1 baris tersimpan)

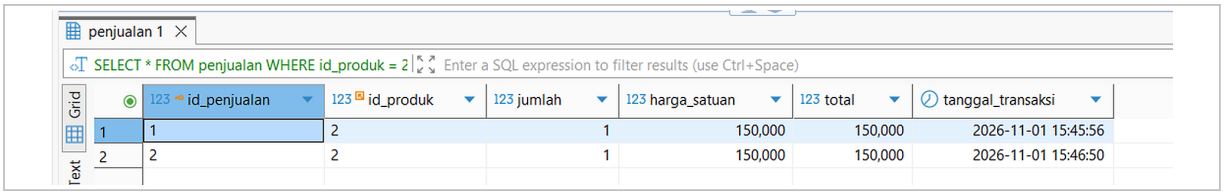


Gambar 6. Skenario B: INSERT 100 unit Mouse ditolak (stok tidak mencukupi)



Gambar 7. Skenario C: INSERT 0 unit Mouse ditolak (jumlah harus lebih dari nol)

Tabel penjualan hanya memuat transaksi yang diterima, sedangkan skenario B dan C tidak tersimpan. Pada pengujian ini skenario A dieksekusi dua kali sehingga tersimpan dua baris transaksi Mouse dengan jumlah 1 unit.



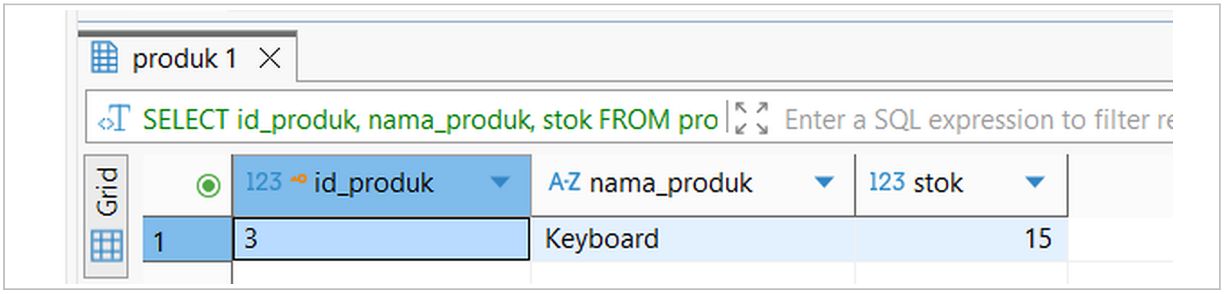
Gambar 8. Data tabel penjualan untuk produk Mouse

4.4 Tugas 4. Pembaruan dan Pencatatan Stok

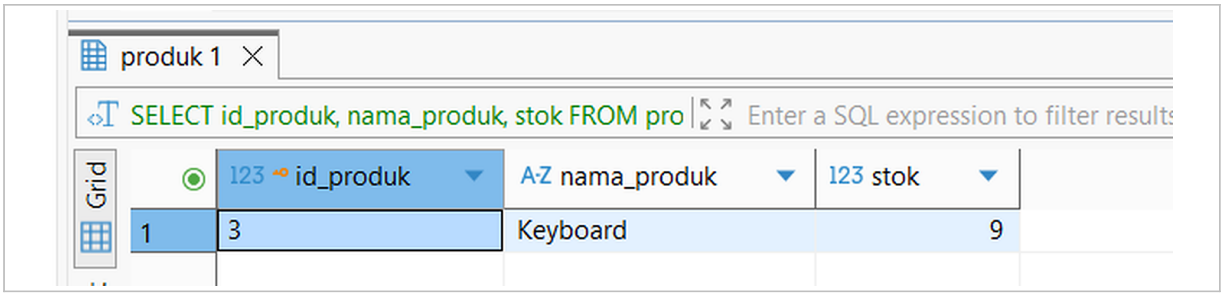
Trigger update_stok_penjualan diuji melalui transaksi pembelian tiga unit Keyboard.

```
INSERT INTO penjualan (id_produk, jumlah, harga_satuan, total)
VALUES (3, 3, 350000, 0);
```

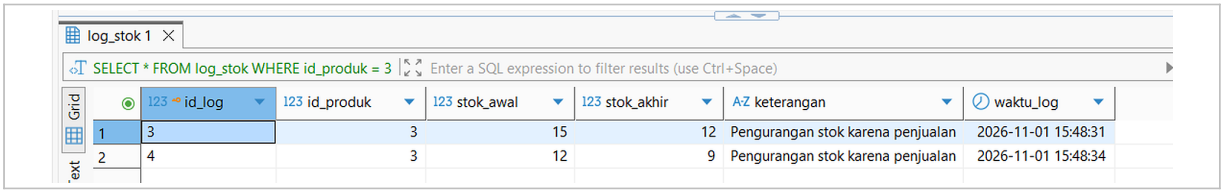
Stok awal Keyboard adalah 15 unit (Gambar 9). Setelah transaksi pertama, stok menjadi 12 unit dan perubahan tersebut tercatat pada log_stok dengan stok awal 15 dan stok akhir 12 (id_log 3, Gambar 11). Transaksi ini dieksekusi dua kali pada pengujian, sehingga eksekusi kedua mengurangi stok dari 12 menjadi 9 (id_log 4) dan stok akhir yang tampil pada Gambar 10 adalah 9 unit.



Gambar 9. Stok Keyboard sebelum transaksi



Gambar 10. Stok Keyboard sesudah transaksi



Gambar 11. Isi tabel log_stok untuk produk Keyboard

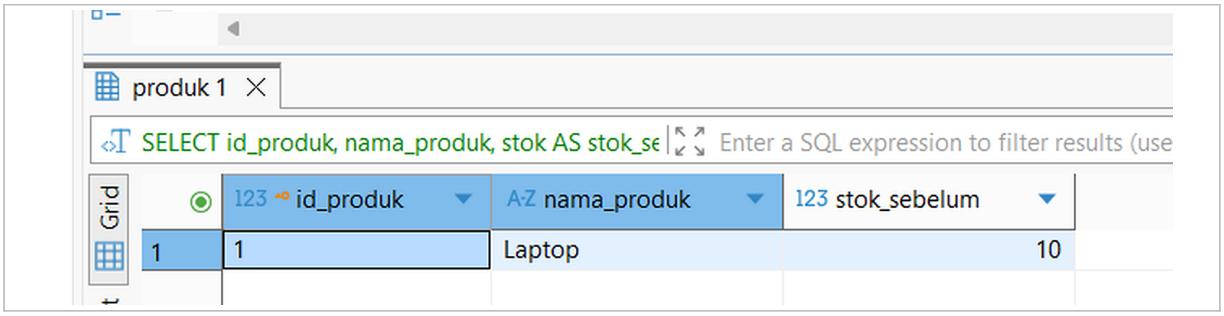
4.5 Tugas 5. Integrasi Function dan Trigger

Pengujian integrasi dilakukan melalui transaksi pembelian dua unit Laptop.

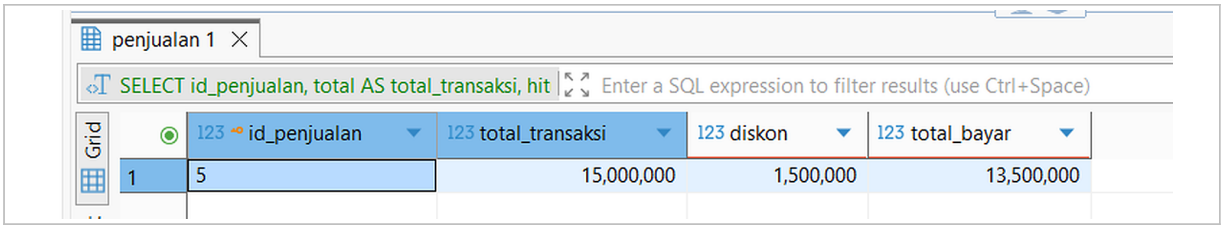
```
INSERT INTO penjualan (id_produk, jumlah, harga_satuan, total)
VALUES (1, 2, 7500000, 0);

SELECT id_penjualan, total AS total_transaksi,
      hitung_diskon(total) AS diskon,
      total - hitung_diskon(total) AS total_bayar
FROM penjualan
WHERE id_produk = 1
ORDER BY id_penjualan DESC
LIMIT 1;
```

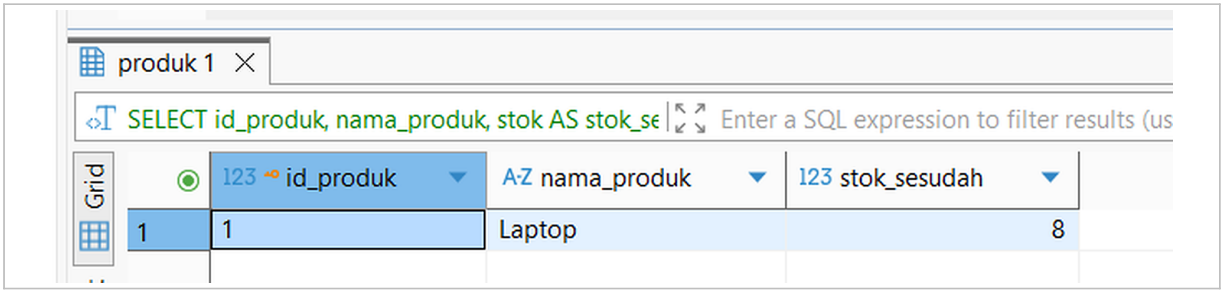
Komponen	Hasil
Total transaksi	Rp15.000.000
Diskon (10%)	Rp1.500.000
Total pembayaran setelah diskon	Rp13.500.000
Stok sebelum transaksi	10 unit
Stok sesudah transaksi	8 unit
Catatan log stok	stok_awal 10, stok_akhir 8 (id_log 5)



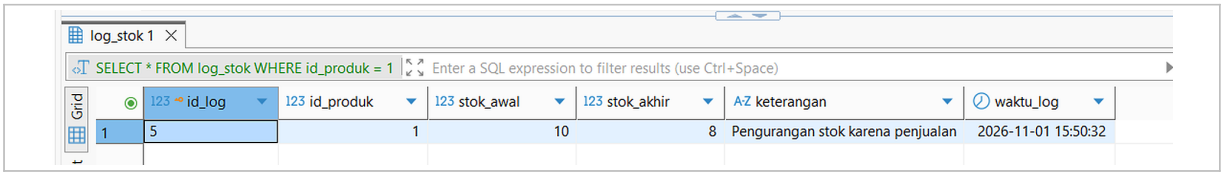
Gambar 12. Stok Laptop sebelum transaksi



Gambar 13. Total transaksi, diskon, dan total pembayaran transaksi Laptop



Gambar 14. Stok Laptop sesudah transaksi



Gambar 15. Isi tabel log_stok untuk produk Laptop

5. Analisis

Function hitung_total dan hitung_diskon menghasilkan nilai yang sesuai dengan rumus. Diskon 10% hanya diberikan pada transaksi minimal Rp1.000.000, sehingga transaksi Rp800.000 tidak mendapat diskon, sedangkan transaksi Rp1.500.000 mendapat diskon Rp150.000 dengan total pembayaran Rp1.350.000.

Trigger validasi_penjualan berhasil menolak transaksi dengan jumlah melebihi stok dan transaksi dengan jumlah nol. Pesan kesalahan yang muncul sesuai dengan pesan yang didefinisikan pada trigger, dan transaksi yang ditolak tidak tersimpan pada tabel penjualan. Transaksi valid diterima dan kolom total terisi otomatis dari function hitung_total.

Trigger update_stok_penjualan mengurangi stok secara otomatis setelah transaksi berhasil dan mencatat stok awal serta stok akhir ke tabel log_stok. Hal ini terlihat pada transaksi Keyboard (15 menjadi 12) dan Laptop (10 menjadi 8). Eksekusi ganda pada transaksi Keyboard juga tercatat sebagai dua baris log yang berurutan (15 ke 12, lalu 12 ke 9), sehingga riwayat perubahan stok dapat ditelusuri.

Pada transaksi Laptop, function dan trigger bekerja bersama: function menghitung total dan diskon, sedangkan trigger memvalidasi transaksi, memperbarui stok, dan mencatat log. Dengan demikian aturan bisnis dijalankan secara otomatis dan konsisten sehingga integritas data tetap terjaga.

6. Kesimpulan

- Function hitung_total dan hitung_diskon dapat dipakai untuk menghitung total transaksi dan diskon secara konsisten.

- Trigger BEFORE INSERT validasi_penjualan efektif menolak transaksi tidak valid sebelum data disimpan.
- Trigger AFTER INSERT update_stok_penjualan memperbarui stok dan mencatat riwayat perubahan stok secara otomatis.
- Kombinasi function dan trigger membantu menjaga integritas data pada sistem penjualan toko elektronik.